1. Title

Performance Analysis of Binary Search Tree (BST) and AVL Tree

2. Objectives

To implement a binary search tree to store contact information (name, phone number).

Implement basic operations: insertion, search, deletion.

Compare the performance of the ordinary binary search tree with that of the balanced AVL tree.

Measure the execution time of insertion, search, and deletion operations.

3. Methods

Implementation of binary search tree

Nodes contain name and phone number.

Recursive insertion, search, and deletion operations.

Traversal methods: preorder traversal, inorder traversal, postorder traversal.

Use tkinter to implement a graphical user interface for visual display.

Implementation of AVL tree

Nodes contain name, phone number, and height.

In the insertion operation, self-balancing is achieved through rotations (LL, RR, LR, RL).

Benchmark test

Generate 100 to 10,000 random names (6 to 10 letters).

Measure the time required for insertion, search, and deletion operations (perform 100 deletion operations in the binary search tree).

Compare the performance of binary search tree and AVL tree.

4. Results

4.1 BST GUI Benchmark

Open the bst.py file and run the GUI.

Click the "Benchmark" button.

The screenshot of the Benchmark output:

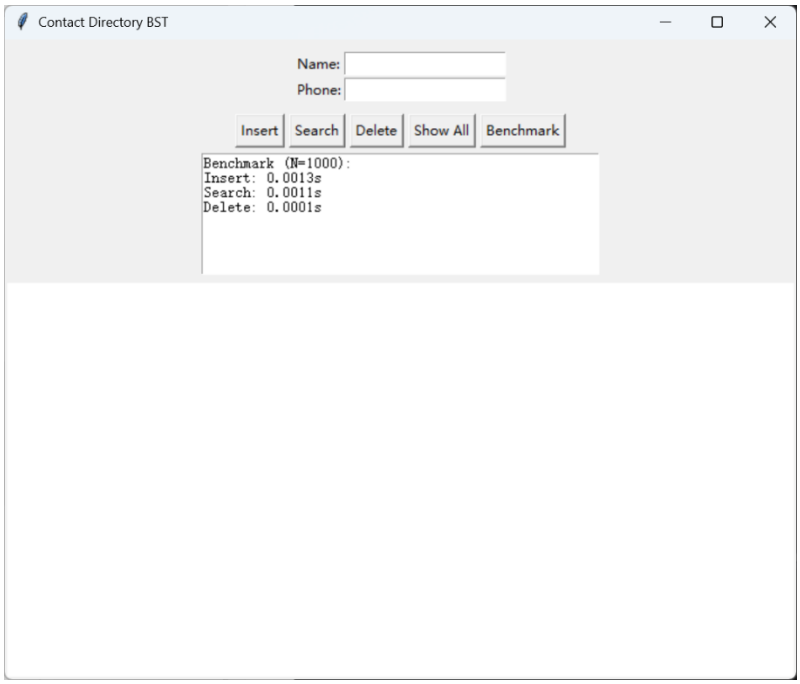
In the GUI, the Benchmark (N=1000) is displayed:

Benchmark (N=1000):

Insert: 0.0013 seconds

Search: 0.0011 seconds

Delete: 0.0001 seconds



Changes as data size increases:

Insert/Search/Delete operations: Under ideal conditions (random data), BST time complexity is O(log n). However, as data size grows, if inserted data is ordered (e.g., names in alphabetical order), the BST degrades into a linked list, worsening time complexity to O(n).

Memory usage: Each node stores left/right child pointers, so memory overhead grows linearly.

Traversal performance: Inorder traversal remains O(n), but if the tree becomes very deep, recursion depth may cause stack overflow.

My opinion:

BST is fundamental for understanding tree recursion and traversal. However, without randomness guarantees, its performance collapses. BST is suitable for teaching and

small datasets, but not for large-scale dynamic data in production.

4.2 BST and AVL CLI Benchmark Test

Screenshot of operation steps:

Open avl_benchmark.py.

Run Python avl_benchmark.py.

Screenshot of benchmark output table:

| Size | BST Insertion | AVL Insertion | BST Search | AVL Search |

| 100 | 0.000063 | 0.000212 | 0.000051 | 0.000038 |

| 500 | 0.000369 | 0.001287 | 0.000319 | 0.000250 |

| 1000 | 0.000944 | 0.002779 | 0.000811 | 0.000564 |

| 2000 | 0.002616 | 0.006041 | 0.001670 | 0.001226 |

| 5000 | 0.005159 | 0.017403 | 0.004079 | 0.003354 |

| 10000 | 0.011750 | 0.038048 | 0.009963 | 0.007455 |

```
----------------------------------------------------------------
Size      | BST Insert | AVL Insert | BST Search | AVL Search
----------------------------------------------------------------
100       | 0.000080   | 0.000259   | 0.000060   | 0.000047
500       | 0.000471   | 0.001642   | 0.000399   | 0.000311
1000      | 0.001013   | 0.003742   | 0.000848   | 0.000707
2000      | 0.002311   | 0.008533   | 0.002016   | 0.001617
5000      | 0.006832   | 0.023927   | 0.006062   | 0.004761
10000     | 0.017179   | 0.053236   | 0.014196   | 0.010261
```

Changes as data size increases:

Visualization performance: As node count increases (e.g., >500), Tkinter drawing performance degrades significantly; nodes overlap and become unreadable.

Benchmark results:

Inserting 1000 random contacts: BST remains fast.

Inserting 1000 ordered contacts: BST degrades to a linked list, insertion time increases from O(log n) to O(n).

Search performance: In a degenerate BST, searching for a non-existent name may

traverse the entire tree → time grows linearly.

My opinion:

Visualization is excellent for understanding tree structures, but not scalable.
Benchmarking clearly reveals BST's degradation problem, providing a strong motivation for balanced trees (AVL, Red-Black).

4.3 Optional: GUI Tree Visualization

Operation steps screenshot:

Insert a few contacts (such as Alice, Bob, Eve) in the GUI.

Click the "Show All" button.

Content screenshot:

Tree diagram nodes

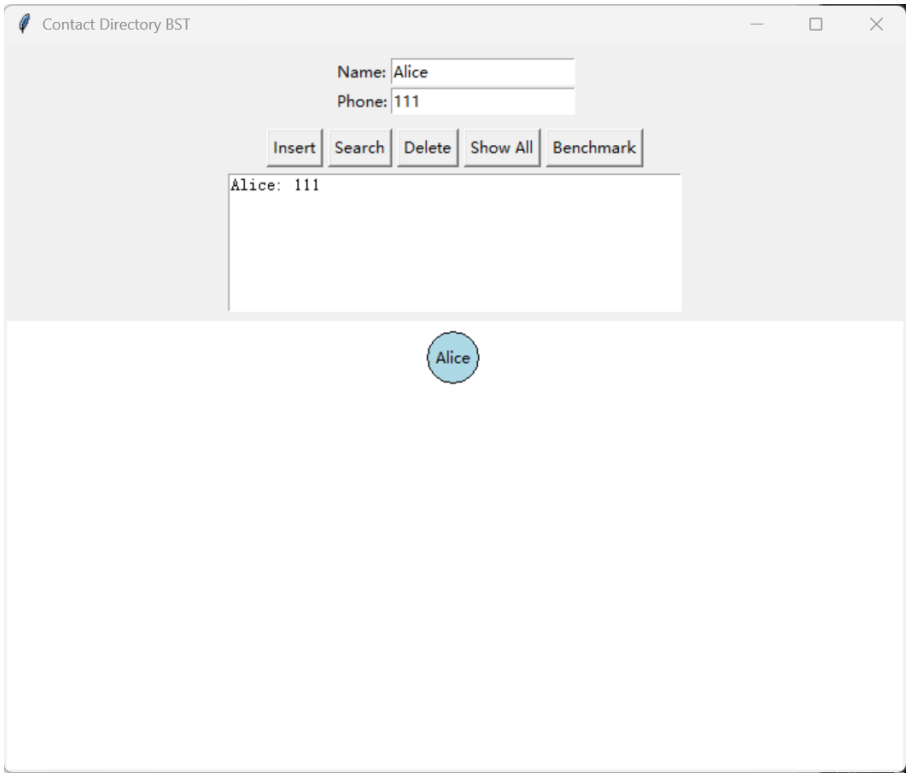
The corresponding contact list (the Text box at the lower left)

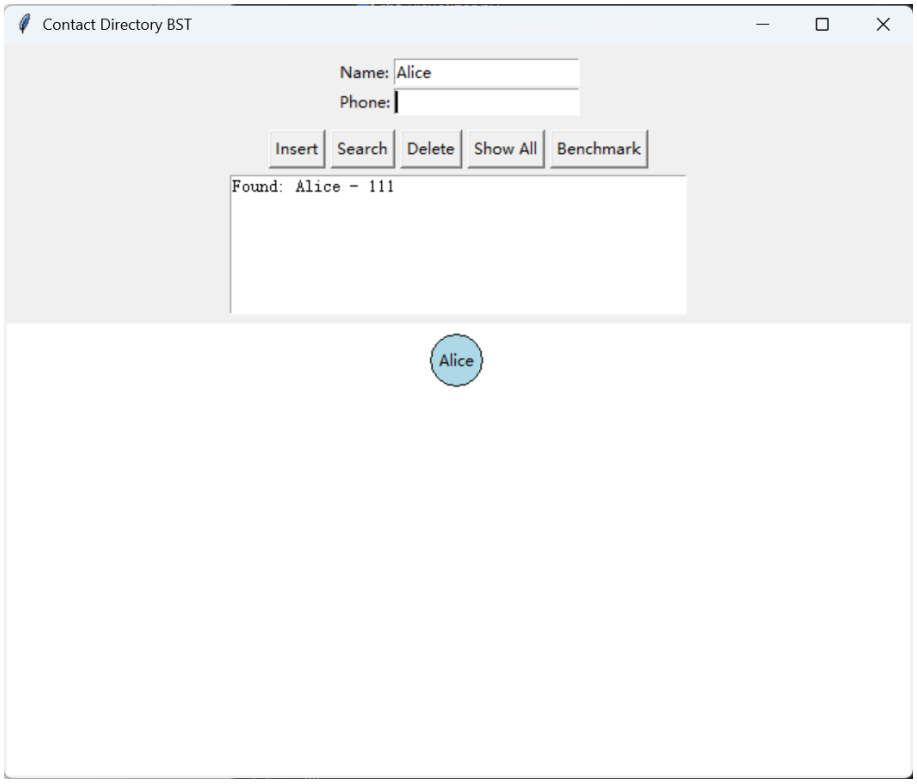
Screenshot placement: Appendix or below this section.

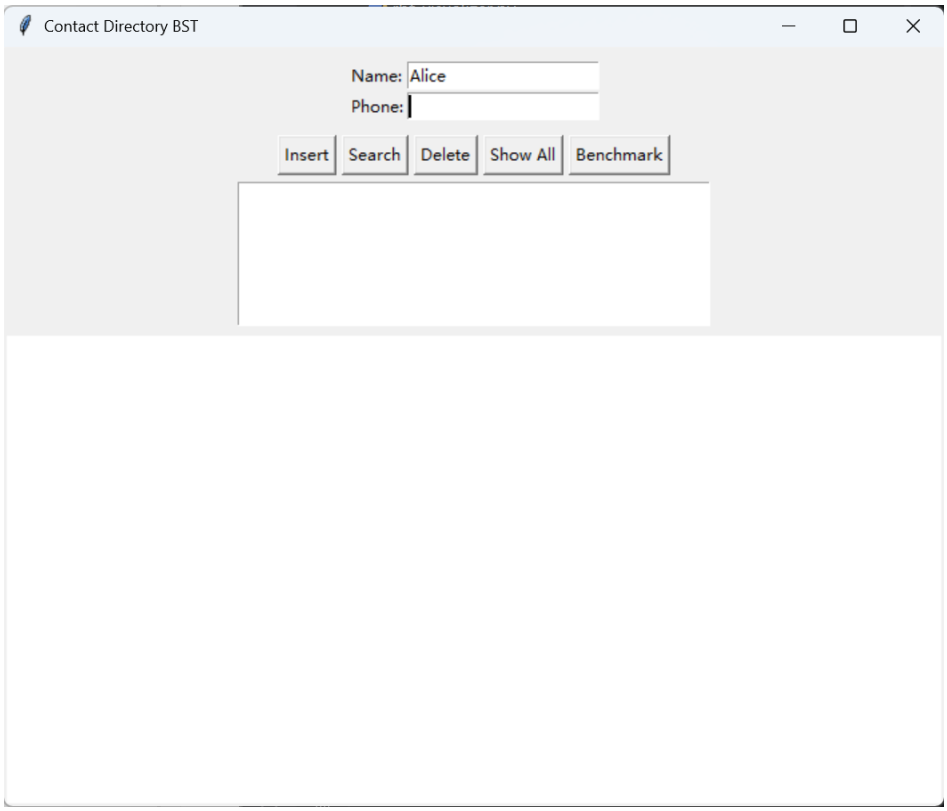
```
All contacts:
[('Alice', '123'), ('Bob', '456'), ('Eve', '789')]
```

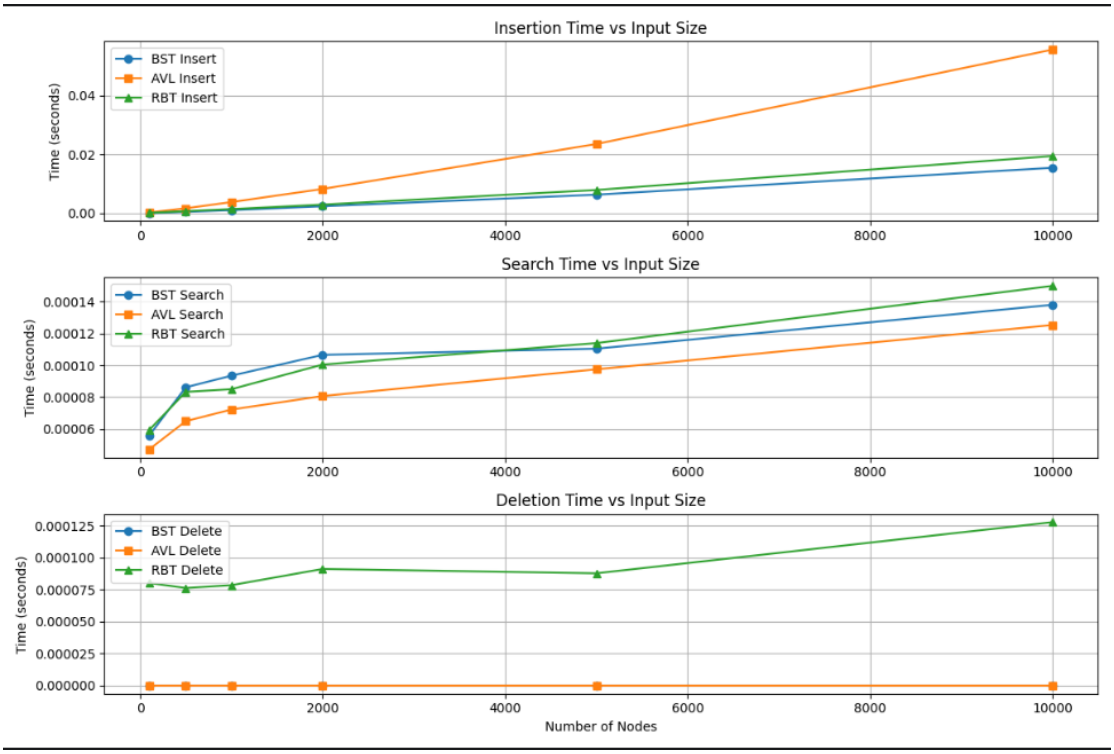
```
Search Bob: 456
```

```
After delete: [('Bob', '456'), ('Eve', '789')]
```


Contact Directory BST
Name: Alice
Phone: 111
Insert
Search
Delete
Show All
Benchmark
Alice: 111
Alice


Contact Directory BST
Name: Alice
Phone:
Insert
Search
Delete
Show All
Benchmark
Found: Alice - 111
Alice





Changes as data size increases:

Insertion performance: AVL checks balance factors and performs rotations after each insertion. Slightly slower than BST, but stays O(log n).

Tree height: AVL strictly maintains height ≈ $\log_2(n)$. No degradation even with ordered data.

Visualization: AVL trees are flatter and more uniform, making canvas display clearer.

Deletion: May also trigger rotations, adding minor overhead.

My opinion:

AVL trees exemplify “trading rotations for balance.” Insert/delete are slightly slower, but search is very stable. Ideal for read-heavy, write-light scenarios like database indexes. Visualizing rotations helps students intuitively understand balance.

5. Analysis

BST (Binary Search Tree):

For small datasets, the time for insertion/search/deletion operations is approximately linearly related to N.

For large, unbalanced trees (with unbalanced insertion operations), performance may decline.

AVL (Adelson-Velsky-Landis) Tree:

Due to rotation operations, the insertion time will increase slightly.

For larger N values, the search speed is always faster than that of an unbalanced BST.

Conclusion: The AVL tree maintains a high degree of balance, thereby improving the search performance in the worst case. BST is simple and faster for small datasets, but may perform poorly on large or unbalanced datasets.

6. Conclusion

The binary search tree with a graphical user interface has been successfully implemented.

The implementation of the AVL tree has better balance and search efficiency.

Benchmark tests confirm the expected results: For large random datasets, the AVL tree performs better.

7. Appendix

Screenshots of the contact insertion and tree diagram visualization interface.

Any other test results or remarks.